



James' Super Cool IS 201 Cheat Sheet

You



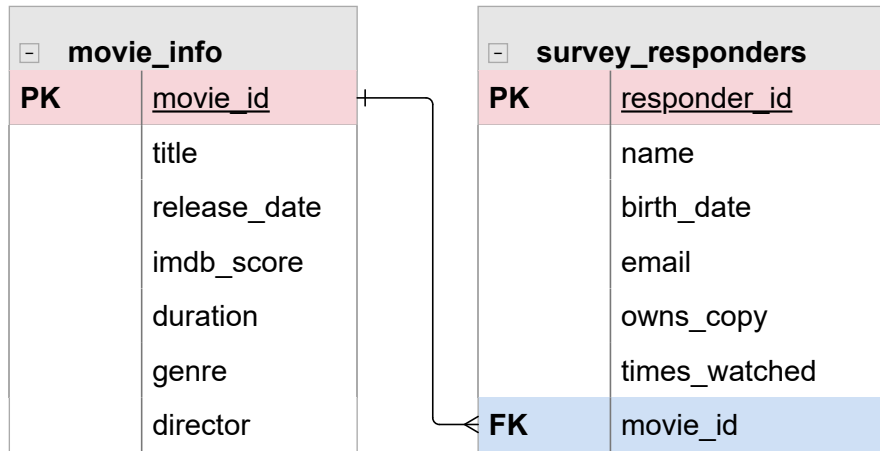


Index

Slide	Topic
2	 <u>ERDs</u>
6	 <u>SQL</u>
12	 <u>Flow Charts</u>
13	 <u>VBA</u>
19	 <u>Statistics</u>
20	 <u>Tableau</u>
21	 <u>Solver</u>
22	 <u>HTML</u>
31	 <u>CSS</u>



ERD: Primary / Foreign Keys



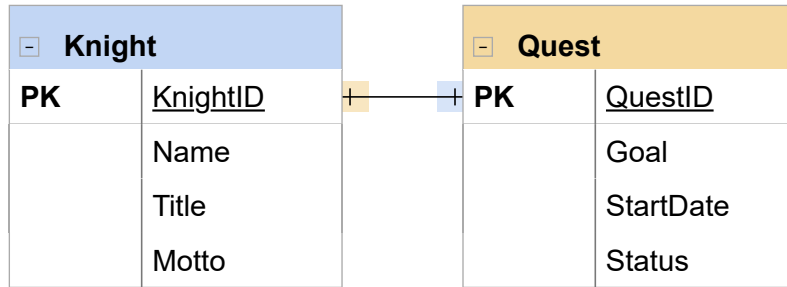
A **primary key** is a unique identifier (think social security number)

A **foreign key** is another table's primary key, used to link two tables together

Entity
Relationship
Diagram



ERD: Cardinality

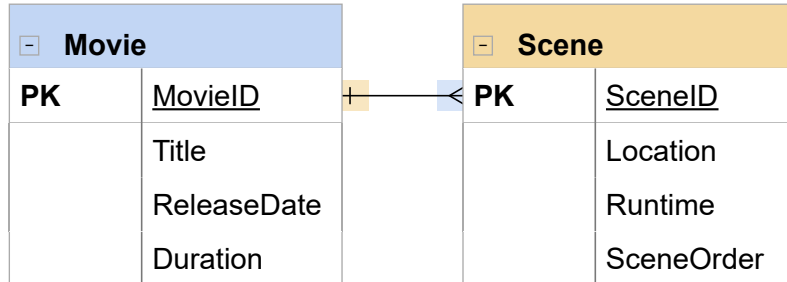


Read left-to-right AND right-to-left

One to One (1:1)

A knight has one quest

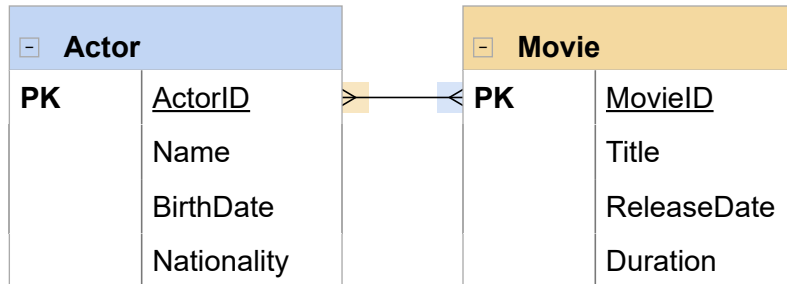
A quest belongs to one knight



One to Many (1:N)

A movie has multiple scenes

A scene belongs to one movie



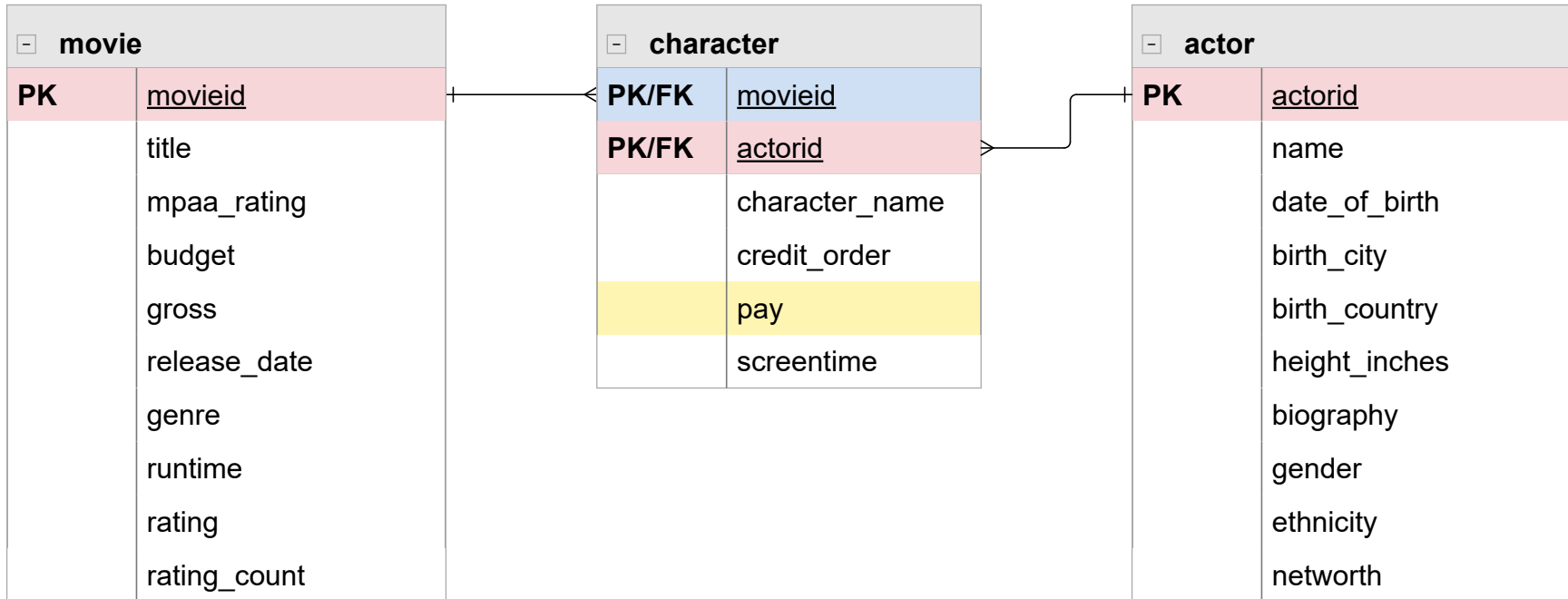
Many to Many (M:N)

An actor can be in many movies

A movie can have lots of actors

Note! M:N cardinality requires a composite table (next slide)

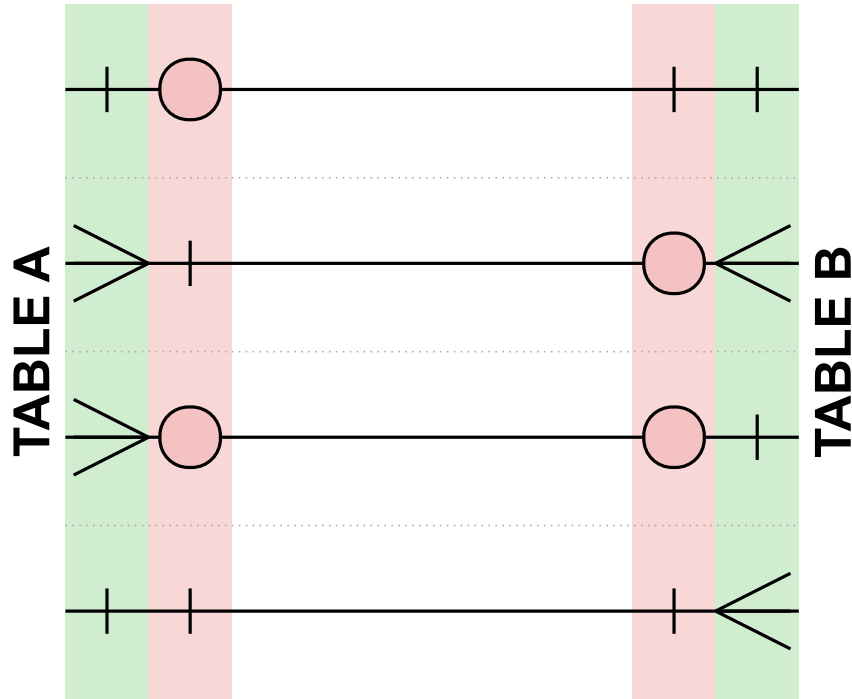
ERD: Composite Table



To know how much money Graham Chapman got **paid** to play King Arthur in *Monty Python and the Holy Grail*, you need to know both the **movie** and the **actor**. His pay was probably different when he played Brian in *Life of Brian*

ERD: Minimum / Maximum Cardinality

The inner marks are minimum (0 or 1)
and the outer marks are maximum (1 or many)



A could have 1 B or 1 B

B could have 0 A or 1 A

A could have 0 B or many B

B could have 1 A or many A

A could have 0 B or 1 B

B could have 0 A or many A

A could have 1 B or many B

B could have 1 A or 1 A

Think: a knight could have 0 horses (coconuts!) or multiple.



go to gaskination.com/sql

SQL: Clauses

Always written in this order. Only SELECT and FROM are required.

- 1 **SELECT** -- which attributes to show (* = all, DISTINCT = no duplicate rows)
- 2 **FROM** -- the table to pull from (tableA)
- 3 **JOIN** -- tableB ON tableA.key = tableB.key [see Join slides](#)
- 4 **WHERE** -- keep only rows that pass a filter [see Where slides](#)
- 5 **AND/OR** -- add more filters (only write WHERE once)
- 6 **GROUP BY** -- with aggregates: list every SELECT attribute that isn't aggregated
- 7 **HAVING** -- like WHERE, but filters groups
- 8 **ORDER BY** -- sort by an attribute: ASC (default, low to high) or DESC
- 9 **LIMIT** -- only show the first N rows (e.g. LIMIT 10)

```
-- aggregates turn many rows into one value
-- function(attribute) AS new_name
SELECT avg(imdb_score) AS avg_score
-- examples: count(), sum(), avg(), min(), max()

-- you can also do math with attributes (+ - * /)
SELECT imdb_score / duration AS score_per_min
```

Structured
Query
Language



SQL: Where (1/2)

Text

in quotes

name = "James"
→ exactly "James"

name != "James"
name <> "James"
→ anything but "James"

title LIKE "%Grail%"
→ contains "Grail"

% = any number of characters
_ = exactly one character

name IN ("James", "Ava", "Sam")
→ exact match for any of these

Blank

any data type

movie_id IS NULL
→ blank / missing

movie_id IS NOT NULL
→ has data



SQL: Where (2/2)

Numbers

no quotes

`imdb_score > 8`

`imdb_score < 8`

→ greater than / less than

`duration BETWEEN 90 AND 120`

→ 90 to 120 minutes (includes both)

`imdb_score >= 8`

`imdb_score <= 8`

→ ... or equal to

Booleans

no quotes

`owns_copy = TRUE`

→ people who own the movie

`owns_copy = FALSE`

→ everyone else

Dates

in quotes:

`"YYYY-MM-DD"`

`release_date > "1980-01-01"`

→ released after 1980

`birth_date < "2000-01-01"`

→ born before 2000

`release_date BETWEEN "1970-01-01" AND "1979-12-31"`

→ released in the 1970s



SQL: Join (1/2)

JOIN *combines two tables, keeping rows that match in both*

```
SELECT survey_responders.name, movie_info.title
FROM survey_responders
JOIN movie_info
  ON survey_responders.movie_id = movie_info.movie_id
```

ON says which columns have to match. Each survey_responders row is paired with the movie_info row that has the same movie_id.

survey_responders

name	movie_id
James	1
Ava	2
Sam	3

movie_info

movie_id	title
1	Monty Python...
2	Princess Bride
4	Shrek



result

name	title
James	Monty Python...
Ava	Princess Bride

Sam (movie 3) and Shrek (movie 4) have no match, so they're left out.

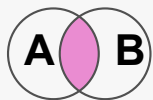


SQL: Join (2/2)

A = the FROM table, B = the joined table; same example as the last slide.

INNER JOIN

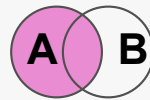
only matches (same as just JOIN)



name	title
James	Monty Python...
Ava	Princess Bride

LEFT JOIN

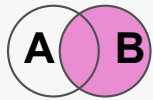
all of A + matches from B



name	title
James	Monty Python...
Ava	Princess Bride
Sam	NULL

RIGHT JOIN

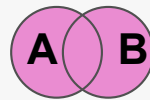
all of B + matches from A



name	title
James	Monty Python...
Ava	Princess Bride
NULL	Shrek

FULL OUTER JOIN

everything from both



name	title
James	Monty Python...
Ava	Princess Bride
Sam	NULL
NULL	Shrek

Missing matches show up as **NULL** (that's where IS NULL comes in handy).



SQL: Example

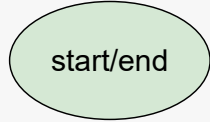
Question: *For well-rated (7+) movies with more than one fan: how many years after each movie came out were its fans born, on average?*

```
SELECT movie_info.title,  
       count(*) AS fans,  
       avg(survey_responders.birth_date - movie_info.release_date) AS avg_years_between  
FROM survey_responders  
JOIN movie_info ON survey_responders.movie_id = movie_info.movie_id  
WHERE movie_info.imdb_score >= 7  
      AND survey_responders.birth_date IS NOT NULL  
GROUP BY movie_info.title  
HAVING count(*) > 1  
ORDER BY avg_years_between DESC  
LIMIT 5
```

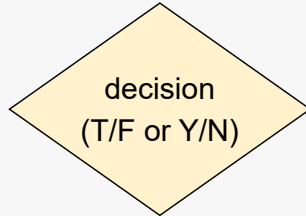
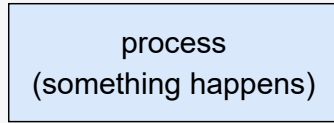


Flow Charts

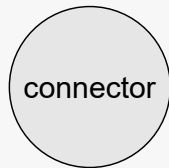
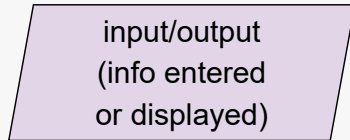
The shapes



you can only have **ONE** start
but you **CAN** have multiple
ends

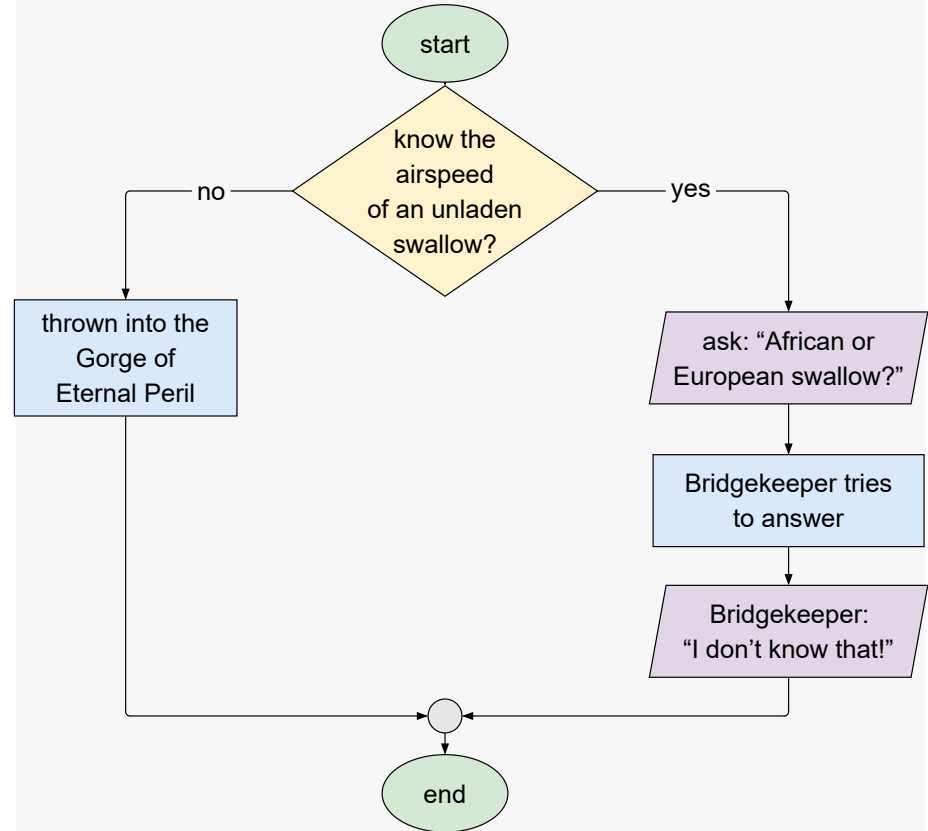


decisions are the **ONLY**
thing that can have more
than one arrow pointing **OUT**
of them



connectors are the **ONLY**
thing that can have more
than one arrow pointing
INTO them

Example: crossing the Bridge of Death



Make yours at draw.io → when done: **File** → **Export as** → **PDF**



VBA: Basics

Option Explicit 'This makes it so that you can only use variables you've declared (VERY RECOMMENDED)

```
Sub thisIsMySubName()
```

```
'BASICS!
```

```
'Generally you will first refer to an object type (like a Sheet)
```

```
'and then tell it what you want to do (like delete or add or copy, etc):
```

```
' Sheets and Worksheets are the same thing
```

```
Sheets("MySheet").Delete
```

```
Sheets.Add.name = "MySheet"
```

```
Worksheets("MySheet").Activate
```

```
' Range and Cells are SIMILAR but different: Range("E7") = Cells(7, 5)
```

```
Range("A1").Activate
```

```
ActiveCell.Value = "Ni!"
```

```
Range("A1:D3").Copy
```

```
Range("E5").PasteSpecial
```

```
' Columns and Rows are what they sound like
```

```
Columns("B:D").Delete
```

Visual
Basic for
Applications



VBA: Navigation and Misc.

'NAVIGATION!

```
'
ActiveCell.End (xlDown)
ActiveCell.End (xlToLeft)

' Move (rows, columns) based on a cell or range
ActiveCell.Offset(1, 2).Value = "NewSpot"
    Range("A1").Offset(3, 4).Value = "NewSpot" 'The cell that now says "NewSpot" is E4
```

' MISCELLANEOUS!

```
' Bolding
Range("A1").Font.Bold = True

' Autofitting columns
Columns.AutoFit 'all columns
Columns("A:C").AutoFit 'specific columns

' Currency format
Range("A1").Style = "Currency"

' Bottom border
Range("A1").Borders(xlEdgeBottom).LineStyle = xlContinuous

' Text splicing
name = "James"
first_initial = Left(name, 1)
```



VBA: Variables

```
'DECLARE VARIABLES!
Dim i As Integer    'a whole number
Dim j As Double     'a number with a decimal
Dim k As String     'a line of text
Dim l As Boolean    'true or false
Dim m As Worksheet

i = 5
j = 1.2345
k = "Tis but a scratch!"
l = True
Set m = Sheets("Sheet1") 'objects (sheets, books, ranges) use Set instead of =

'USER INTERFACE!

'Tell the user something
MsgBox "Halt! Who would cross the Bridge of Death?"

'Ask for an input
Dim username As String
username = InputBox("What is your name?")

' COMBINING STRINGS

Dim myOutput As String
Dim name As String
name = "James"

' COMbine strings with ampersands (&)-- don't forget spaces
myOutput = "My name is " & name & " and I seek the Grail!"
```



'CONDITIONALS!

VBA: Conditionals

```
' If Statements
If i < 0 Then
    ' your code here
ElseIf i < 5 Then
    ' your code here
ElseIf i < 10 Then
    ' your code here
Else
    ' your code here
End If
```

```
'Case Statements
Select Case fruit
    Case "apple", "strawberry"
        Color = "red"
    Case "coconut"
        Color = "brown"
    Case Else
        Color = "unknown"
End Select
```

```
'Another Case Statement
Select Case imdb_score
    Case Is >= 8
        verdict = "must watch (like Monty Python)"
    Case 6 To 8
        verdict = "pretty good"
    Case Is < 6
        verdict = "I'm probably skipping this one"
End Select
```



'LOOPS!

VBA: Loops

```
'For Loop
For i = 0 To 5
    ' your code here
Next i
```

```
'For Each Loop
Dim x As Range
For Each x In Range("A2", Cells(lastrow, "A"))
    ' your code here
Next
```

```
'For Each Loop 2
Dim x As Range
Dim mtRange As Range
Set myRange = Range("A1:E5")
For Each x In myRange
    ' your code here
Next
```

```
'Do While Loop
Do While i = 5 'condition does NOT have to be numbers
    ' your code here
Loop
```

```
' Do Until Loop
Do Until i = 10 'condition does NOT have to be numbers
    'your code here
Loop
```

End Sub



VBA: Functions

```
Function thisIsMyFunctionName(num As Integer, tf As Boolean) As String
```

```
' a function declaration is just like a set of dims  
' the INPUT is in the parentheses ()  
' the OUTPUT is outside of it  
' so this function KIND OF has this baked into it:
```

```
Dim num As Integer
```

```
Dim tf As Boolean
```

```
Dim thisIsMyFunctionName As String
```

```
' the INPUT then comes from your excel sheet  
' for example, in your sheet you cell might say:  
' =thisIsMyFunctionName(15, True)
```

```
' the OUTPUT comes from your function  
' set the function's name equal to something  
' in this case, our output has to be a string:
```

```
thisIsMyFunctionName = "Your mother was a hamster!"
```

```
' generally your inputs will be used to calculate/decide your output
```

```
End Function
```



Statistics

TEST & USE CASE

Correlation

numeric – numeric
(Pearson r coefficient)

Regression

numeric – multi. numeric

ANOVA

numeric – mult. categorical

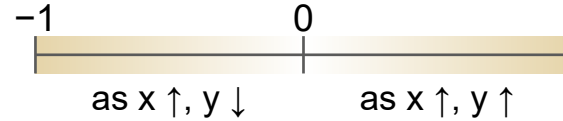
T-test

numeric – 2 pair categorical

OUTPUT

HOW TO READ IT

→ r



strength = distance from 0; sign = direction

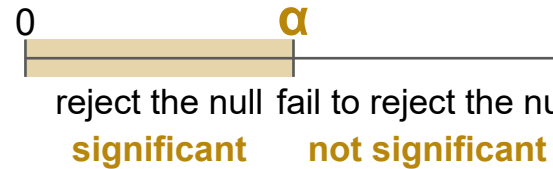
→ R^2



70% of the variance in y
is explained by the x 's

usually 0.05

→ p



*p works like a percentage:
0.05 = 5% to see this or more extreme*

Null Hypothesis

H_0 : No x 's are
significant

Alt Hypothesis

H_A : At least one x is
significant

Scientific Notation

$aE^b = a \times 10^b$

$aE^3 = a \times 1000$ (big)

$aE^{-3} = a \times 0.001$
(small)

Tableau

“Default” means leave it alone

If the assignment says “default,” you don’t have to change anything. It’s the setting Tableau starts with.

Toolbar buttons worth knowing



Swap rows and columns: flips the x- and y-axis. A sideways chart can tell a clearer story.



Sort ascending: smallest to largest



Sort descending: largest to smallest



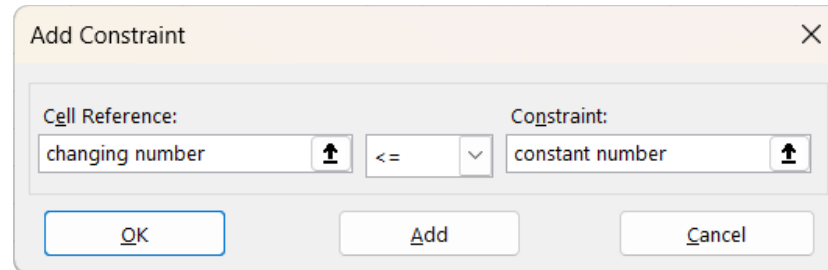
Solver

Two important Excel formulas:

=sum(A1:A5) → $A1 + A2 + A3 + A4 + A5$

=sumproduct(A1:A5, B11:B15) → $A1*B11 + A2*B12 + \dots + A5*B15$
the two ranges should be the same size and shape

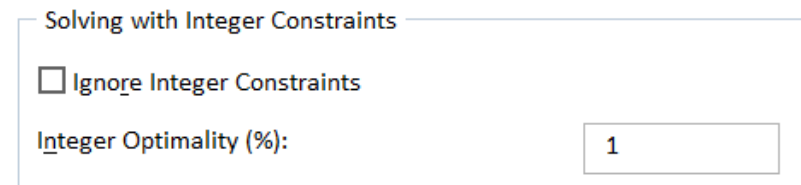
Constraints:



The 'Add Constraint' dialog box in Excel. It has a title bar 'Add Constraint' with a close button. Inside, there are two input fields: 'Cell Reference:' containing 'changing number' and 'Constraint:' containing 'constant number'. Between them is a dropdown menu showing '<=' with a small arrow icon. Below the input fields are three buttons: 'OK', 'Add', and 'Cancel'.

If integer isn't working:

Solver > Options > uncheck this box:



The 'Solving with Integer Constraints' section of the Solver Options dialog. It contains a checkbox labeled 'Ignore Integer Constraints' which is currently unchecked. Below it is a label 'Integer Optimality (%)' followed by a text box containing the number '1'.



HTML: Setup

In an html file inside VS Code,
type an exclamation point and enter to automatically generate:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>

</body>
</html>
```

Go here for free bootstrap themes:
startbootstrap.com/themes/portfolio-resume

HyperText
Markup
Language



HTML: Tags

```
<!-- This is a comment -->
<h1>The Holy Grail</h1>
<h2>The Knights</h2>
<h3>Sir Robin</h3>

<p>Bravely ran away.</p>

<!-- Line Break -->
<br>
```

```
<!-- Unordered List (Bullets) -->
<ul>
  <li>Sir Lancelot</li>
  <li>Sir Galahad</li>
  <li>Sir Robin</li>
</ul>

<!-- Change <ul> to <ol>
for an ordered list (Numbers) -->
```

Every tag follows this pattern:

<tag>	← start tag
stuff	← what the tag applies to
</tag>	← end tag: same name, plus a /

The start and end tags act like parentheses.

(a few, like
, have no end tag)



HTML: Anchors

```
<!-- Hyperlinks -->
```

```
<a href="https://www.google.com">Visit Google</a>
```

```
<!-- Folder References -->
```

```
<a href="otherpage.html">Visit Other Page</a>
```

```
<a href="subfolder/page.html">Visit Subfolder Page</a>
```

```
<a href="../anotherpage.html">Visit Another Page</a> ← ../ goes up one folder
```

```
<!-- On-page Anchors -->
```

```
<h1 id="top">This is the main heading</h1>
```

```
<a href="#top">Jump to the top</a> ← # + the id jumps to it on the same page
```

HTML: Images

```
<!-- Images -->



```

Notice: images do NOT have an end tag `</img>`
(neither do line breaks `</br>`)



HTML: Divisions

Divs don't inherently do anything: they're invisible boxes you put content into so that you can isolate it for positioning and CSS styling using classes.

HTML:

```
<div class="square-image">
  
</div>
```

CSS:

```
.square-image {
  width: 200px;
  height: 200px;
}
```

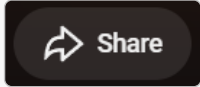
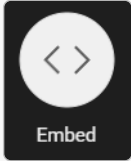
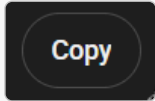
For more, see:

[CSS: Selectors/Properties/Values](#)

HTML: Embedding

Embeds sometimes don't show up until your site is live

YouTube

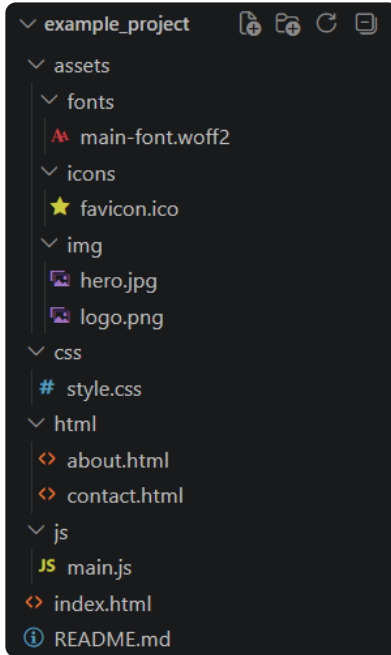
- 1 Under the video, click **Share** 
- 2 Click **Embed** 
- 3 Click **Copy** 
- 4 Paste it into your HTML

Tableau

- 1 Go to public.tableau.com/app/discover
- 2 Search for a relevant viz
- 3 Click the share icon 
- 4 Click **Copy Embed Code** 
- 5 Paste it into your HTML



HTML: Best Practices



Use folders!

Give each type of file its own folder: `css/`, `html/`, `js/`, and `assets/` (with `img/`, `fonts/`, `icons/` inside).
`index.html` stays at the top level. It's your homepage, so it's the first page the site loads.

Never use spaces in file or folder names

Use a dash or underscore instead:

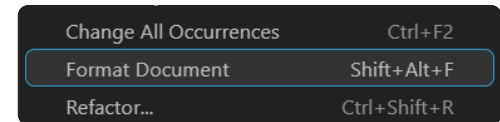
✓ `about-me.html` ✗ `about me.html`
✓ `example_project/` ✗ `example project/`

Format your code

Fixes indentation and long lines.

Right click → **Format Document**

shortcut: `Shift+Alt+F`





HTML: Uploading to GitHub

A. Create the repository

1 Go to github.com and sign in (or make a free account)

2 Click **New**



3 Type a **Repository name** (no spaces!)

Repository name *

example_project

example_project is available.

4 Leave it **Public** and turn **Add README** on

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

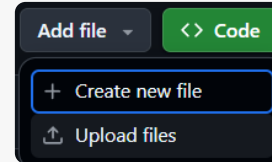
On ☒

5 Click **Create repository** at the bottom



B. Upload your website

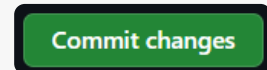
6 On your new repository's page, click **Add file** → **Upload files**



Open your project folder and drag everything *inside* it (index.html, css/, ...)

7 **Upload the CONTENTS of your project, not the folder itself**

8 Click **Commit changes** ("commit" = save)



Next: turn it into a live website → [Going Live](#)

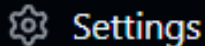


HTML: Going Live

On your repository's page (from the last slide):

1

Click the **Settings** tab at the top



Settings

2

In the left sidebar, click **Pages**



Codespaces



Pages

3

Under **Branch**, change None to main, then click **Save**



main



/(root)

Save

4

Wait a couple of minutes, then **refresh** the page. Your website's link will appear at the top:

Your site is live at https://jimna-h.github.io/example_project/

Last **deployed** by jimna-h 1 minute ago

Visit site

Unpublish site



CSS: Basics

1. Connect it to your HTML

```
<head>
  <link rel="stylesheet" href="css/style.css">
</head>
```

Goes inside the `<head>` of every page.
`href` is the path to your `.css` file.

2. Every rule looks like this

```
selector {
  property: value;
  property2: value2;
}
```

property = what to change, **value** = what to change it to. That's all CSS is! (see next slide)

3. It's cascading: the later rule wins

```
p {
  color: blue;
}
p {
  color: red;
}
```

If you paint a door blue and then re-paint it red, it will be red instead of blue. CSS works the same way, but with everything not just colors.

Cascading
Style
Sheets



CSS: Selectors / Properties / Values

Built-in selectors

Use a tag name (body, h1, p, a, img, ...) to style every one of those tags.

```
body { ← every <body>
  background-color: #f0f0f0;
  font-family: Arial, sans-serif;
  margin: 0;
  padding: 20px;
}
```

Class (custom) selectors

Make your own: a **period** + any name.
Styles only tags with that class.

```
.highlight {
  background-color: yellow;
  font-weight: bold;
}
```

Apply it in your HTML (no period):

```
<p class="highlight">Ni!</p>
```

There are A LOT of selectors and properties. Google and AI are your friend for finding exactly what you want!



jimna-h.github.io/james_super_cool_is_201_cheatsheet



Suggestions? Reach out to me at jbruce1@byu.edu



IS 201 TA Lab